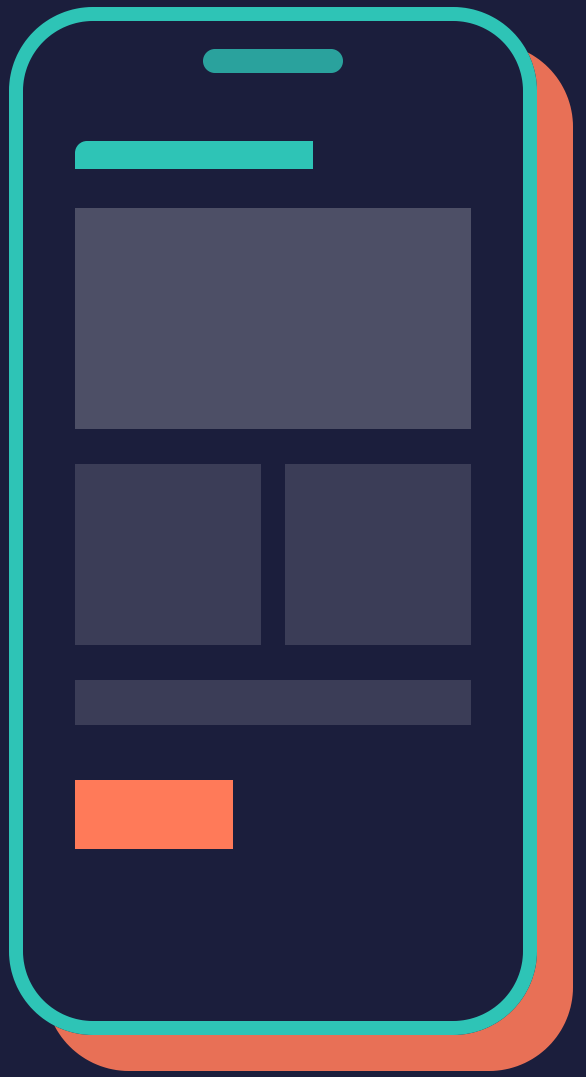
Navigation et cycle de vie

Cours 3 – Expo Router, données distantes et listes

Chris CHEVALIER



Sommaire

01 Naviguer sur mobile et sur le web

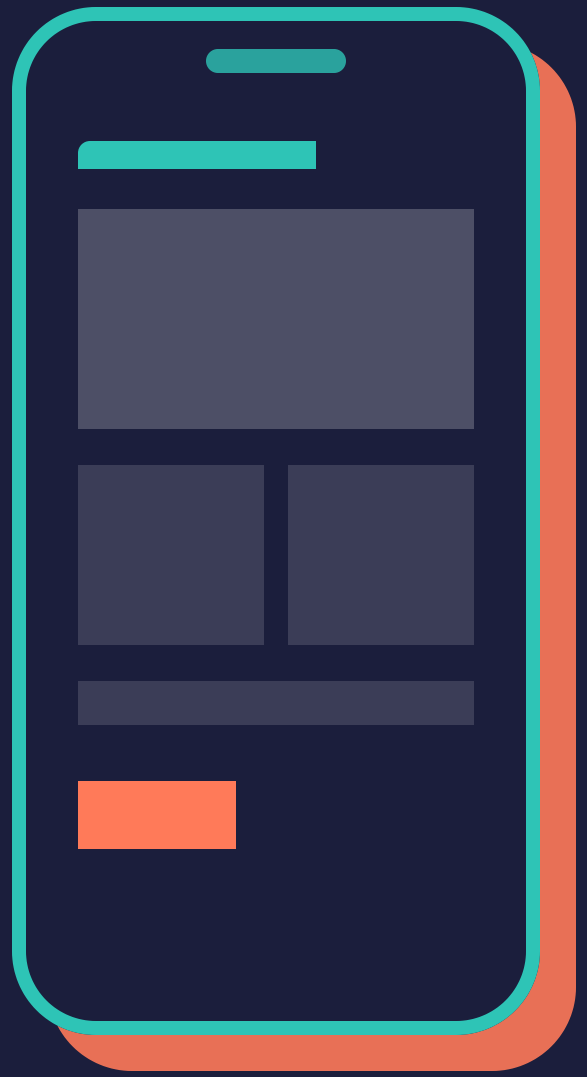
02 Expo Router : les fichiers sont les routes

03 Onglets, pile et routes dynamiques

04 Une deuxième API : la météo

05 Le cycle de vie

06 Les listes volumineuses



01

Naviguer sur mobile et sur le web

Deux logiques à réconcilier



Deux façons de naviguer

	Sur le web	Sur mobile
Ce qui définit l'écran	L'URL dans la barre d'adresse	Une **pile** d'écrans en mémoire
Le retour	Le bouton du navigateur	La flèche de l'en-tête, le geste ou le bouton système
Les sections	Un menu	Des **onglets** en bas de l'écran
Ouvrir un contenu de l'extérieur	Un lien partagé	Un **lien profond** (deep link) qui ouvre l'application sur le bon écran

> Avec **Expo Router**, chaque écran a une URL. La même navigation produit une pile sur mobile et des adresses dans le navigateur.

La pile d'écrans

1. Onglet « Séjours »

`/`

Le catalogue est la base de la pile

2. On empile

`/stays/s3`

Le détail se pose par-dessus, avec une flèche de retour

3. On dépile

`/`

Le catalogue réapparaît **dans l'état où on l'avait laissé** (filtres, défilement)

> Expo Router s'appuie sur React Navigation, la bibliothèque de navigation de référence. Vous n'avez pas à la configurer vous-mêmes.

02

Expo Router : les fichiers sont les routes

Le routage par fichiers



Un fichier, une URL

src/app/

```
src/app/
├── _layout.tsx          pile racine
├── +not-found.tsx       URL inconnue
├── (tabs)/
│   ├── _layout.tsx      onglets
│   ├── index.tsx        /
│   └── favorites.tsx    /favorites
└── stays/
    └── [id].tsx         /stays/s1, /stays/s2…
```

- `_layout.tsx` : le **navigateur** (pile ou onglets) de son dossier
- `(tabs)` : un **groupe**. Les parenthèses n'apparaissent pas dans l'URL
- `[id]` : un **segment dynamique**, récupéré comme paramètre
- `+not-found` : l'écran des adresses inconnues

Préparer le projet

Terminal

```
npx expo install @expo/vector-icons
mkdir "src/app/(tabs)"
mv src/app/index.tsx "src/app/(tabs)/"
```

- Les icônes des onglets viennent de `@expo/vector-icons`. `npx expo install` choisit la version compatible avec le SDK
- Le catalogue devient le premier onglet : on le range dans le groupe `(tabs)`
- Sous Windows, déplacez le fichier dans l'explorateur de VS Code

La pile racine

src/app/_layout.tsx

```tsx
import { Stack } from "expo-router";

import { FavoritesProvider } from "@/context/favorites-context";

// Ouvert directement par son URL, un séjour garde un retour vers les
// onglets
export const unstable_settings = { anchor: "(tabs)" };

export default function RootLayout() {
  return (
    <FavoritesProvider>
      <Stack screenOptions={{ headerTransparent: false }}>
        <Stack.Screen
          name="(tabs)"
          options={{ headerShown: false, title: "Séjours" }}
        />
      </Stack>
    </FavoritesProvider>
  );
}
```

- Les onglets forment **un seul écran** de la pile, sans en-tête (ils ont le leur). Son `title` sert au bouton retour du détail, qui afficherait sinon « (tabs) »
- Le détail se pose par-dessus
- `anchor` : sans lui, une page ouverte par son URL n'a pas de flèche de retour

03

Onglets, pile et routes dynamiques

Construire le parcours de l'utilisateur



Le deuxième onglet : les favoris

src/app/(tabs)/favorites.tsx

```tsx
import { FlatList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";

export default function FavoritesScreen() {
  const { state } = useStays();
  const { isFavorite } = useFavorites();

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const favoriteStays = state.stays.filter((stay) =>
    isFavorite(stay.id),
  );

  return (
    <FlatList
      data={favoriteStays}
      keyExtractor={(stay) => stay.id}
```

- Un nouvel écran dans `src/app/(tabs)/` : il s'ouvre à l'adresse `/favorites`
- Il lit les favoris dans le Context, comme le badge du catalogue
- La liste affichée se **calcule** à partir des séjours et des favoris : aucun état en double

Le deuxième onglet : les favoris (suite)

src/app/(tabs)/favorites.tsx

```tsx
      renderItem={({ item }) => (
        <Text style={styles.item}>
          {item.title} · {item.city}
        </Text>
      )}
      ListEmptyComponent={
        <Text style={styles.item}>Aucun favori pour le moment.</Text>
      }
      contentContainerStyle={styles.list}
    />
  );
}

const styles = StyleSheet.create({
  list: { padding: 16, gap: 8 },
  item: { fontSize: 17 },
});
```

Les onglets

src/app/(tabs)/_layout.tsx

```tsx
import Ionicons from "@expo/vector-icons/Ionicons";
import { Tabs } from "expo-router/js-tabs";

const ACTIVE = "#0B7A70";
const INACTIVE = "#8C90B4";

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: ACTIVE,
        tabBarInactiveTintColor: INACTIVE,
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: "Séjours",
          tabBarIcon: ({ focused, size }) => (
            <Ionicons
```

- Depuis le SDK 58, importez `Tabs` depuis `expo-router/js-tabs`. L'import depuis `expo-router` est **déprécié**
- `name` reprend le nom du fichier
- Icônes : `@expo/vector-icons`, à installer avec `npx expo install`

Les onglets (suite)

src/app/(tabs)/_layout.tsx

```
            name="bed-outline"
            size={size}
            color={focused ? ACTIVE : INACTIVE}
          />
        ),
      }}
    />
    <Tabs.Screen
      name="favorites"
      options={{
        title: "Favoris",
        tabBarIcon: ({ focused, size }) => (
          <Ionicons
            name="heart-outline"
            size={size}
            color={focused ? ACTIVE : INACTIVE}
          />
        ),
      }}
    />
```

Les onglets (suite)

src/app/(tabs)/_layout.tsx

```
41      </Tabs>
42    );
43  }
```

Une page introuvable : +not-found

src/app/+not-found.tsx

```tsx
import { Link, Stack } from "expo-router";
import { StyleSheet, Text, View } from "react-native";

export default function NotFoundScreen() {
  return (
    <View style={styles.box}>
      <Stack.Screen options={{ title: "Page introuvable" }} />
      <Text style={styles.text}>Cette page est introuvable.</Text>
      <Link href="/" style={styles.link}>
        Retour aux séjours
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  box: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
```

- Expo Router affiche cet écran pour toute adresse inconnue
- `<Link href="/">` ramène à l'accueil
- Testez sur le web avec une adresse inventée, par exemple `/vacances`

Une page introuvable : +not-found (suite)

src/app/+not-found.tsx

```
    gap: 12,
  },
  text: { fontSize: 18 },
  link: { fontSize: 16, color: "#0B7A70", fontWeight: "600" },
});
```

Charger un seul séjour

src/services/stays-service.ts

```ts
import { Stay, StayType } from "@/models/stay";

const SITE = "https://chris-chevalier.github.io/react-native-supports";
const STAYS_URL = `${SITE}/api/stays.json`;

// Forme de la réponse de l'API, limitée à ce qu'on utilise
type StayResponse = {
  id: string;
  title: string;
  type: StayType;
  price_per_night: number;
  is_available: boolean;
  image_url: string | null;
  location: { city: string; latitude: number; longitude: number };
};

// Traduit une réponse de l'API dans le vocabulaire du modèle
function toStay(response: StayResponse): Stay {
  return {
    id: response.id,
```

- Le détail ne reçoit qu'un identifiant : le service retrouve le séjour
- Il renvoie `undefined` si l'identifiant est inconnu. L'écran devra le prévoir

Charger un seul séjour (suite)

src/services/stays-service.ts

```
    title: response.title,
    city: response.location.city,
    type: response.type,
    pricePerNight: response.price_per_night,
    available: response.is_available,
    latitude: response.location.latitude,
    longitude: response.location.longitude,
    imageUrl: response.image_url ?? undefined,
  };
}

export async function fetchStays(): Promise<Stay[]> {
  const response = await fetch(STAYS_URL);
  if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`);
  const data: StayResponse[] = await response.json();
  return data.map(toStay);
}

export async function fetchStay(
  id: string,
```

Charger un seul séjour (suite)

src/services/stays-service.ts

```ts
): Promise<Stay | undefined> {
  const stays = await fetchStays();
  return stays.find((stay) => stay.id === id);
}
```

Le hook useStay

src/hooks/use-stay.ts

```ts
import { useEffect, useState } from "react";

import { Stay } from "@/models/stay";
import { fetchStay } from "@/services/stays-service";

export type StayState =
  | { status: "loading" }
  | { status: "not-found" }
  | { status: "success"; stay: Stay };

export function useStay(id: string) {
  const [state, setState] = useState<StayState>({
    status: "loading",
  });

  useEffect(() => {
    let cancelled = false;
    fetchStay(id).then((stay) => {
      if (!cancelled)
        setState(
```

- Trois états : chargement, introuvable, succès
- `id` est une dépendance de l'effet : changer de séjour recharge les données

Le hook useStay (suite)

src/hooks/use-stay.ts

```
          stay
            ? { status: "success", stay }
            : { status: "not-found" },
        );
    });
    return () => {
      cancelled = true;
    };
  }, [id]);

  return state;
}
```

Une route dynamique

src/app/stays/[id].tsx

```tsx
import { Stack, useLocalSearchParams } from "expo-router";
import {
  Image,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { StateMessage } from "@/components/state-message";
import { useStay } from "@/hooks/use-stay";
import { Stay } from "@/models/stay";

export default function StayScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const state = useStay(id);

  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "not-found")
    return <StateMessage message="Ce séjour est introuvable." />;
```

- `useLocalSearchParams` lit `id` dans l'URL
- L'écran ne reçoit **que l'identifiant**, jamais l'objet entier : il recharge ses données. C'est ce qui permet d'ouvrir la page depuis un lien
- `<Stack.Screen options>` change le titre de l'en-tête
- Un identifiant inconnu est un cas prévu

Une route dynamique (suite)

src/app/stays/[id].tsx

```tsx
  return <StayDetails stay={state.stay} />;
}

function StayDetails({ stay }: { stay: Stay }) {
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Stack.Screen options={{ title: stay.title }} />
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>
          {stay.city} · {stay.pricePerNight} € / nuit
        </Text>
      </View>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  content: { width: "100%", maxWidth: 800, alignSelf: "center" },
```

Une route dynamique (suite)

src/app/stays/[id].tsx

```
41    image: { width: "100%", aspectRatio: 16 / 9 },
42    body: { padding: 16, gap: 8 },
43    title: { fontSize: 26, fontWeight: "800" },
44    city: { fontSize: 17, color: "#5A5F80" },
45  });
```

URL et liens profonds

Sur le web

- L'URL `/stays/s3` s'affiche dans la barre d'adresse
- On peut la copier, la partager, la recharger : le même écran s'ouvre
- Le bouton retour du navigateur dépile

Sur mobile

- Le `scheme` d' `app.json` définit le préfixe : `cours3://stays/s3`
- Dans Expo Go, l'adresse devient `exp://<IP>:8081/--/stays/s3`
- Une notification ou un e-mail peut ouvrir l'application sur le bon séjour

Un paramètre d'URL vient de l'extérieur : **ne lui faites pas confiance**. Vérifiez qu'il existe avant de l'utiliser, comme le cas `not-found` du détail.

Naviguer avec un lien : `Link`

src/app/(tabs)/favorites.tsx

```tsx
import { Link } from "expo-router";
import { FlatList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";

export default function FavoritesScreen() {
  const { state } = useStays();
  const { isFavorite } = useFavorites();

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const favoriteStays = state.stays.filter((stay) =>
    isFavorite(stay.id),
  );

  return (
    <FlatList
      data={favoriteStays}
```

- `<Link>` est l'équivalent de `<a href>` : sur le web, c'est un vrai lien
- `href` reçoit le chemin de la route et ses paramètres : ici l'identifiant du séjour
- Les **routes typées** (`typedRoutes` dans `app.json`) font signaler par TypeScript un lien vers une page inexistante

Naviguer avec un lien : Link (suite)

src/app/(tabs)/favorites.tsx

```tsx
      keyExtractor={(stay) => stay.id}
      renderItem={({ item }) => (
        <Link
          href={{ pathname: "/stays/[id]", params: { id: item.id } }}
          style={styles.item}
        >
          {item.title}
        </Link>
      )}
      ListEmptyComponent={
        <Text style={styles.item}>Aucun favori pour le moment.</Text>
      }
      contentContainerStyle={styles.list}
    />
  );
}

const styles = StyleSheet.create({
  list: { padding: 16 },
  item: { fontSize: 17, paddingVertical: 8 },
```

Naviguer avec un lien : `Link` (suite)

src/app/(tabs)/favorites.tsx

```
41 });
```

Une carte qui s'ouvre au toucher

src/components/stay-card.tsx

```tsx
import {
  Image,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { Stay } from "@/models/stay";

type StayCardProps = {
  stay: Stay;
  isFavorite: boolean;
  onToggleFavorite: (id: string) => void;
  onPress?: (id: string) => void;
};

const formatPrice = (price: number) =>
  new Intl.NumberFormat("fr-FR", {
    style: "currency",
```

- Toute la carte devient une zone d'appui. Le bouton favori garde sa propre action
- `onPress` est facultative : une carte sans action reste utilisable ailleurs
- `onPress?.(stay.id)` n'appelle la fonction que si elle existe

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```
    currency: "EUR",
    maximumFractionDigits: 0,
  }).format(price);

export function StayCard({
  stay,
  isFavorite,
  onToggleFavorite,
  onPress,
}: StayCardProps) {
  return (
    <Pressable onPress={() => onPress?.(stay.id)} style={styles.card}>
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>{stay.city}</Text>
        <Text style={styles.price}>
          {formatPrice(stay.pricePerNight)} / nuit
        </Text>
        {!stay.available && (
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```tsx
          <Text style={styles.unavailable}>Complet</Text>
        )}
        <Pressable
          onPress={() => onToggleFavorite(stay.id)}
          style={({ pressed }) => [
            styles.button,
            pressed && styles.buttonPressed,
          ]}
        >
          <Text style={styles.buttonLabel}>
            {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
          </Text>
        </Pressable>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```
    flex: 1,
    backgroundColor: "#fff",
    borderRadius: 12,
    overflow: "hidden",
    margin: 8,
  },
  image: { width: "100%", aspectRatio: 4 / 3 },
  body: { padding: 12, gap: 4 },
  title: { fontSize: 18, fontWeight: "700" },
  city: { fontSize: 15, color: "#5A5F80" },
  price: { fontSize: 16, color: "#0B7A70" },
  unavailable: { fontSize: 14, color: "#B42318" },
  button: {
    marginTop: 8,
    paddingVertical: 10,
    borderRadius: 8,
    backgroundColor: "#E2F7F4",
    alignItems: "center",
  },
  buttonPressed: { opacity: 0.6 },
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```
81    buttonLabel: { fontSize: 15, fontWeight: "600" },
82  });
```

Naviguer par le code : `useRouter`

src/app/(tabs)/index.tsx

```tsx
import { useRouter } from "expo-router";
import { FlatList, StyleSheet, Text, View } from "react-native";

import { StateMessage } from "@/components/state-message";
import { StayCard } from "@/components/stay-card";
import { StayFiltersBar } from "@/components/stay-filters-bar";
import { useFavorites } from "@/hooks/use-favorites";
import { useColumns } from "@/hooks/use-columns";
import { useStayFilters } from "@/hooks/use-stay-filters";
import { useStays } from "@/hooks/use-stays";

export default function CatalogScreen() {
  const router = useRouter();
  const columns = useColumns();
  const { state, reload } = useStays();
  const stays = state.status === "success" ? state.stays : [];
  const { filters, updateFilters, visibleStays } =
    useStayFilters(stays);
  const { isFavorite, toggleFavorite } = useFavorites();
```

- Pour naviguer après une action (un appui, un envoi de formulaire), sans lien visible : `useRouter()` renvoie l'objet `router`

Méthode	Effet
`push(href)`	Empile un écran
`back()`	Revient à l'écran précédent
`replace(href)`	Remplace l'écran actuel (après une connexion, par exemple)
`navigate(href)`	Va à l'écran, sans l'empiler s'il existe déjà

Naviguer par le code : useRouter (suite)

src/app/(tabs)/index.tsx

```tsx
const openStay = (id: string) =>
  router.push({ pathname: "/stays/[id]", params: { id } });

if (state.status === "loading") return <StateMessage loading />;
if (state.status === "error")
  return <StateMessage message={state.message} onRetry={reload} />;

return (
  <View style={styles.container}>
    <StayFiltersBar filters={filters} onChange={updateFilters} />
    <FlatList
      key={columns}
      numColumns={columns}
      data={visibleStays}
      keyExtractor={(stay) => stay.id}
      renderItem={({ item }) => (
        <StayCard
          stay={item}
          isFavorite={isFavorite(item.id)}
          onToggleFavorite={toggleFavorite}
```

Naviguer par le code : useRouter (suite)

src/app/(tabs)/index.tsx

```
41              onPress={openStay}
42            />
43          )}
44          ListEmptyComponent={
45            <Text style={styles.empty}>
46              Aucun logement ne correspond.
47            </Text>
48          }
49        />
50      </View>
51    );
52  }
53
54  const styles = StyleSheet.create({
55    container: {
56      flex: 1,
57      width: "100%",
58      maxWidth: 1100,
59      alignSelf: "center",
60      padding: 8,
```

Naviguer par le code : `useRouter` (suite)

src/app/(tabs)/index.tsx

```
61    },
62    empty: { margin: 8, fontSize: 16 },
63  });
```

Quiz

Quelle URL correspond au fichier `src/app/(tabs)/favorites.tsx` ?

> `/favorites` : les groupes entre parenthèses n'apparaissent pas dans l'URL.

Pourquoi passer l'identifiant du séjour au détail plutôt que l'objet complet ?

> L'URL doit suffire à afficher la page : ouverte depuis un lien ou rechargée, elle n'a que l'identifiant. Le détail recharge donc ses données.

04

Une deuxième API : la météo

Des paramètres dans l'URL, une API dans le détail



Le modèle de la météo

src/models/weather.ts

```ts
export type Weather = {
  temperature: number;
  windSpeed: number;
  code: number;
};

// Codes météo de l'Organisation météorologique mondiale (WMO),
// simplifiés
export function describeWeather(code: number): string {
  if (code === 0) return "Ciel dégagé";
  if (code <= 3) return "Nuageux";
  if (code <= 48) return "Brouillard";
  if (code <= 67) return "Pluie";
  if (code <= 77) return "Neige";
  if (code <= 82) return "Averses";
  return "Orage";
}
```

- Le type décrit la météo **dans notre vocabulaire**, pas dans celui de l'API
- `describeWeather` traduit un code numérique en texte : une règle métier pure

Une deuxième API : Open-Meteo

src/services/weather-service.ts

```ts
import { Weather } from "@/models/weather";

// Forme de la réponse de l'API, limitée à ce qu'on utilise
type OpenMeteoResponse = {
  current: {
    temperature_2m: number;
    wind_speed_10m: number;
    weather_code: number;
  };
};

export async function fetchWeather(
  latitude: number,
  longitude: number,
 ): Promise<Weather> {
  const url =
    "https://api.open-meteo.com/v1/forecast" +
    `?latitude=${latitude}&longitude=${longitude}` +
    "&current=temperature_2m,wind_speed_10m,weather_code";
```

- Même démarche que pour les séjours : le type décrit la réponse avec **ses** noms (`temperature_2m`…)
- **Nouveau** : l'adresse porte des **paramètres** (latitude, longitude, mesures voulues). Le gabarit de chaîne `` `…${latitude}…` `` les insère
- Puis `fetch`, `response.ok` et la conversion en modèle, comme pour les séjours
- Open-Meteo est gratuite et sans clé

Une deuxième API : Open-Meteo (suite)

src/services/weather-service.ts

```
21    const response = await fetch(url);
22    if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`);
23    const data: OpenMeteoResponse = await response.json();
24
25    // On convertit la réponse de l'API en objet du modèle
26    return {
27      temperature: data.current.temperature_2m,
28      windSpeed: data.current.wind_speed_10m,
29      code: data.current.weather_code,
30    };
31  }
```

Le contrôleur : useWeather

src/hooks/use-weather.ts

```ts
import { useEffect, useState } from "react";

import { Weather } from "@/models/weather";
import { fetchWeather } from "@/services/weather-service";

export type WeatherState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; weather: Weather };

// Charge la météo à l'ouverture de l'écran
export function useWeather(latitude: number, longitude: number) {
  const [state, setState] = useState<WeatherState>({
    status: "loading",
  });

  useEffect(() => {
    let cancelled = false;
    fetchWeather(latitude, longitude)
      .then((weather) => {
```

- Le même modèle d'état que pour les séjours : chargement, erreur ou succès
- Le hook appelle le service et tient l'état. Il ne sait pas comment il sera affiché

Le contrôleur : `useWeather` (suite)

src/hooks/use-weather.ts

```
21        if (!cancelled) setState({ status: "success", weather });
22      })
23      .catch(() => {
24        if (!cancelled)
25          setState({
26            status: "error",
27            message: "Météo indisponible",
28          });
29      });
30    return () => {
31      cancelled = true;
32    };
33  }, [latitude, longitude]);
34
35  return state;
36 }
```

La vue : WeatherBadge

src/components/weather-badge.tsx

```tsx
import {
  ActivityIndicator,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { WeatherState } from "@/hooks/use-weather";
import { describeWeather } from "@/models/weather";

type WeatherBadgeProps = { state: WeatherState };

// Vue pure : elle affiche l'état de la météo qu'on lui donne
export function WeatherBadge({ state }: WeatherBadgeProps) {
  return (
    <View style={styles.badge}>
      {state.status === "loading" && <ActivityIndicator />}
      {state.status === "error" && <Text>{state.message}</Text>}
      {state.status === "success" && (
        <Text style={styles.text}>
```

- La vue reçoit l'état en props et affiche chaque cas
- Le service parle à l'API, le hook tient l'état, la vue affiche ce qu'on lui donne

La vue : WeatherBadge (suite)

src/components/weather-badge.tsx

```tsx
          {describeWeather(state.weather.code)} ·{" "}
          {Math.round(state.weather.temperature)} °C · vent{" "}
          {Math.round(state.weather.windSpeed)} km/h
        </Text>
      )}
    </View>
  );
}

const styles = StyleSheet.create({
  badge: {
    alignSelf: "flex-start",
    paddingVertical: 8,
    paddingHorizontal: 12,
    borderRadius: 8,
    backgroundColor: "#E2F7F4",
  },
  text: { fontSize: 15, fontWeight: "600" },
});
```

La météo dans le détail

src/app/stays/[id].tsx

```tsx
import { Stack, useLocalSearchParams } from "expo-router";
import {
  Image,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { StateMessage } from "@/components/state-message";
import { WeatherBadge } from "@/components/weather-badge";
import { useStay } from "@/hooks/use-stay";
import { Stay } from "@/models/stay";
import { useWeather } from "@/hooks/use-weather";

export default function StayScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const state = useStay(id);

  if (state.status === "loading") return <StateMessage loading />;
```

- Un composant séparé, `StayDetails` : un hook ne peut pas être appelé après un `return`
- Le détail charge d'abord le séjour, puis la météo de ses coordonnées

La météo dans le détail (suite)

src/app/stays/[id].tsx

```tsx
  if (state.status === "not-found")
    return <StateMessage message="Ce séjour est introuvable." />;
  return <StayDetails stay={state.stay} />;
}

// Composant séparé : les hooks ne peuvent pas être appelés après un
// return
function StayDetails({ stay }: { stay: Stay }) {
  const weather = useWeather(stay.latitude, stay.longitude);

  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Stack.Screen options={{ title: stay.title }} />
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>
          {stay.city} · {stay.pricePerNight} € / nuit
        </Text>
        <WeatherBadge state={weather} />
```

La météo dans le détail (suite)

src/app/stays/[id].tsx

```
41        </View>
42      </ScrollView>
43    );
44  }
45
46  const styles = StyleSheet.create({
47    content: { width: "100%", maxWidth: 800, alignSelf: "center" },
48    image: { width: "100%", aspectRatio: 16 / 9 },
49    body: { padding: 16, gap: 8 },
50    title: { fontSize: 26, fontWeight: "800" },
51    city: { fontSize: 17, color: "#5A5F80" },
52  });
```

Ce qui peut mal se passer

Situation	Ce qui se passe	Ce que voit l'utilisateur
Pas de réseau	`fetch` lève une erreur	Un message et « Réessayer »
Erreur 404 ou 500	`response.ok` vaut `false`	Un message et « Réessayer »
Réponse lente	La promesse attend	Un indicateur de chargement
L'écran est quitté avant la réponse	Le drapeau `cancelled` ignore le résultat	Rien

> Une clé d'API placée dans le code de l'application est **lisible par tous** : le code JavaScript est livré avec l'application. N'y mettez jamais de secret. Préférez des API sans clé ou passez par votre propre serveur.

05

Le cycle de vie

Écrans et application



Un écran ne se démonte pas toujours

src/app/(tabs)/favorites.tsx

```tsx
import { Link, useFocusEffect } from "expo-router";
import { useCallback } from "react";
import { FlatList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";

export default function FavoritesScreen() {
  const { state, reload } = useStays();
  const { isFavorite } = useFavorites();

  // À chaque fois que l'onglet redevient visible, on recharge les
  // séjours
  useFocusEffect(
    useCallback(() => {
      reload();
    }, [reload]),
  );
```

- Un onglet quitté reste **monté** : son `useEffect(…, [])` ne s'exécutera plus
- `useFocusEffect` (d'`expo-router`) s'exécute à **chaque** affichage de l'écran
- La fonction renvoyée, si vous en écrivez une, s'exécute quand l'écran perd le focus
- `useCallback` évite de relancer l'effet à chaque rendu

Un écran ne se démonte pas toujours (suite)

src/app/(tabs)/favorites.tsx

```tsx
if (state.status !== "success")
  return <StateMessage loading={state.status === "loading"} />;
const favoriteStays = state.stays.filter((stay) =>
  isFavorite(stay.id),
);

return (
  <FlatList
    data={favoriteStays}
    keyExtractor={(stay) => stay.id}
    renderItem={({ item }) => (
      <Link
        href={{ pathname: "/stays/[id]", params: { id: item.id } }}
        style={styles.item}
      >
        {item.title}
      </Link>
    )}
    ListEmptyComponent={
      <Text style={styles.item}>Aucun favori pour le moment.</Text>
```

Un écran ne se démonte pas toujours (suite)

src/app/(tabs)/favorites.tsx

```
41        }
42        contentContainerStyle={styles.list}
43      />
44    );
45  }
46
47  const styles = StyleSheet.create({
48    list: { padding: 16 },
49    item: { fontSize: 17, paddingVertical: 8 },
50  });
```

L'application passe en arrière-plan

src/hooks/use-is-app-active.ts

```ts
import { useEffect, useState } from "react";
import { AppState } from "react-native";

// true quand l'application est au premier plan, false en arrière-plan
export function useIsAppActive(): boolean {
  const [isActive, setIsActive] = useState(true);

  useEffect(() => {
    const subscription = AppState.addEventListener(
      "change",
      (status) => setIsActive(status === "active"),
    );
    return () => subscription.remove();
  }, []);

  return isActive;
}
```

- `AppState` signale le premier plan (`"active"`) et l'arrière-plan (`"background"`)
- Sur le web, il suit la visibilité de l'onglet du navigateur
- Toujours **retirer l'écouteur** au démontage, sinon il s'accumule
- `useWeather` s'en sert pour recharger la météo au retour dans l'application

Recharger au retour au premier plan

src/hooks/use-weather.ts

```ts
import { useEffect, useState } from "react";

import { useIsAppActive } from "@/hooks/use-is-app-active";
import { Weather } from "@/models/weather";
import { fetchWeather } from "@/services/weather-service";

export type WeatherState =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "success"; weather: Weather };

// Recharge la météo à l'ouverture de l'écran et à chaque retour au
// premier plan
export function useWeather(latitude: number, longitude: number) {
  const [state, setState] = useState<WeatherState>({
    status: "loading",
  });
  const isActive = useIsAppActive();

  useEffect(() => {
```

- `isActive` devient une dépendance : l'effet se relance quand l'application revient au premier plan
- En arrière-plan, rien n'est chargé

Recharger au retour au premier plan (suite)

src/hooks/use-weather.ts

```
    if (!isActive) return;
    let cancelled = false;
    fetchWeather(latitude, longitude)
      .then((weather) => {
        if (!cancelled) setState({ status: "success", weather });
      })
      .catch(() => {
        if (!cancelled)
          setState({
            status: "error",
            message: "Météo indisponible",
          });
      });
    return () => {
      cancelled = true;
    };
  }, [latitude, longitude, isActive]);

  return state;
}
```

Quel outil pour quel moment ?

Moment	Outil	Exemple
Le composant apparaît pour la première fois	`useEffect(…, [])`	Charger le détail d'un séjour
Un paramètre change	`useEffect(…, [id])`	Recharger quand l'identifiant change
L'écran redevient visible	`useFocusEffect`	Rafraîchir l'onglet des favoris
L'application revient au premier plan	`AppState`	Recharger la météo
Le composant disparaît	la fonction renvoyée par l'effet	Retirer un écouteur, ignorer une réponse tardive

06

Les listes volumineuses

FlatList et SectionList



Une `FlatList` qui reste fluide

Prop ou pratique	Effet
`keyExtractor`	Une clé stable par élément : React réutilise les lignes au lieu de les recréer
`initialNumToRender`	Le nombre d'éléments affichés au premier rendu (10 par défaut)
`windowSize`	Combien d'écrans de contenu sont gardés en mémoire autour de la zone visible
`getItemLayout`	Si toutes les lignes ont la même hauteur, la liste n'a plus à les mesurer
Des éléments légers	Peu de vues imbriquées, des images à la bonne taille
Pas de calcul dans `renderItem`	Filtrer ou trier **avant**, dans le hook

Pour des milliers de lignes, FlashList (Shopify) réutilise encore mieux les éléments.

`SectionList` : regrouper dans le modèle

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  latitude: number;
  longitude: number;
  imageUrl?: string;
};

export type StayFilters = {
  query: string;
  type: StayType | null;
  onlyAvailable: boolean;
};
```

- Chaque section a un `title` et un tableau `data`
- Le regroupement est une **règle du modèle**, pure et testable

`SectionList` : regrouper dans le modèle (suite)

src/models/stay.ts

```ts
export const STAY_TYPE_LABELS: Record<StayType, string> = {
  apartment: "Appartement",
  house: "Maison",
  cabin: "Cabane",
};

// Règle métier : une fonction pure, sans React, facile à tester
export function matchesFilters(
  stay: Stay,
  filters: StayFilters,
 ): boolean {
  const search = filters.query.trim().toLowerCase();
  const matchesQuery =
    stay.title.toLowerCase().includes(search) ||
    stay.city.toLowerCase().includes(search);
  const matchesType =
    filters.type === null || stay.type === filters.type;
  const matchesAvailability =
    !filters.onlyAvailable || stay.available;
  return matchesQuery && matchesType && matchesAvailability;
```

`SectionList` : regrouper dans le modèle (suite)

src/models/stay.ts

```ts
}

export type CitySection = { title: string; data: Stay[] };

// Regroupe des séjours par ville, au format attendu par SectionList
export function groupByCity(stays: Stay[]): CitySection[] {
  const cities = [...new Set(stays.map((stay) => stay.city))].sort();
  return cities.map((city) => ({
    title: city,
    data: stays.filter((stay) => stay.city === city),
  }));
}
```

`SectionList` : afficher les groupes

src/app/(tabs)/favorites.tsx

```tsx
import { Link, useFocusEffect } from "expo-router";
import { useCallback } from "react";
import { SectionList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";
import { groupByCity } from "@/models/stay";

export default function FavoritesScreen() {
  const { state, reload } = useStays();
  const { isFavorite } = useFavorites();

  // À chaque fois que l'onglet redevient visible, on recharge les
  // séjours
  useFocusEffect(
    useCallback(() => {
      reload();
    }, [reload]),
  );
```

- `SectionList` est un composant de base, comme `FlatList` : `sections` remplace son `data`
- `renderSectionHeader` affiche le titre de chaque groupe, `renderItem` chaque élément

`SectionList` : afficher les groupes (suite)

src/app/(tabs)/favorites.tsx

```tsx

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const sections = groupByCity(
    state.stays.filter((stay) => isFavorite(stay.id)),
  );

  return (
    <SectionList
      sections={sections}
      keyExtractor={(stay) => stay.id}
      renderSectionHeader={({ section }) => (
        <Text style={styles.city}>{section.title}</Text>
      )}
      renderItem={({ item }) => (
        <Link
          href={{ pathname: "/stays/[id]", params: { id: item.id } }}
          style={styles.item}
        >
          {item.title}
```

`SectionList` : afficher les groupes (suite)

src/app/(tabs)/favorites.tsx

```tsx
        </Link>
      )}
      ListEmptyComponent={
        <Text style={styles.item}>Aucun favori pour le moment.</Text>
      }
      contentContainerStyle={styles.list}
    />
  );
}

const styles = StyleSheet.create({
  list: { padding: 16 },
  city: {
    fontSize: 14,
    fontWeight: "700",
    textTransform: "uppercase",
    color: "#0B7A70",
    marginTop: 16,
  },
  item: { fontSize: 17, paddingVertical: 8 },
```

`SectionList` : afficher les groupes (suite)

src/app/(tabs)/favorites.tsx

```
61 });
```

Quiz

L'onglet « Favoris » charge ses données dans un `useEffect(…, [])`. J'ajoute un favori depuis le catalogue puis je reviens. Que se passe-t-il ?

> L'onglet n'est pas rechargé : il est resté monté, l'effet ne se relance pas. Il faut `useFocusEffect`.

`fetch` reçoit une réponse 404. Le bloc `catch` s'exécute-t-il ?

> Non : `fetch` ne lève une erreur qu'en cas d'échec réseau. Il faut tester `response.ok` et lever l'erreur soi-même.

À retenir

- Avec Expo Router, **chaque fichier de `src/app/` est une route** et chaque écran a une URL
- `_layout.tsx` choisit le navigateur : pile (`Stack`) ou onglets (`Tabs` depuis `expo-router/js-tabs`)
- Une route dynamique reçoit un identifiant et **recharge** ses données
- Un service appelle l'API avec `fetch`, teste `response.ok` et traduit la réponse en modèle
- `useFocusEffect` pour un écran qui redevient visible, `AppState` pour l'application
- Des listes légères, filtrées en amont. `SectionList` pour des groupes

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

